Reference-Compatible Parallel Execution for Quantized LLM Inference

Chengzhan Li

Abstract—Batching a quantized language model can change its numerical output even when token positions and causal masks stay the same. We introduce reference-derived phase plans for parallel inference that retains an established deployment's numerical behavior. A plan binds the prompt producer, physical KV range, arithmetic recipe and ownership of outputs and state. It places cuts around whole graphs or within attention, while packing independent arithmetic groups into wider operations. Implementations on CUDA and Vulkan cover Qwen, Llama and SmolLM2, including a mixed-Q4 32B model. On Linux/A5000, compatible speculative applications reduce resident-service time by 17.21%, 25.54% and 11.52% against the fastest measured fixed target-only control for Qwen3-8B, Qwen3-14B and Llama-8B. Separate balanced experiments attribute 0.80%–1.64% to Q8 projection packing; local cuts contribute smaller, geometry-dependent gains. For 32B mixed quantization, independent-channel grouping reduces ordinary service time by 5.61% relative to a graph-level scalar-column loop. Eight-query execution also lowers aggregate cost at equal work, while longer live proposals add enough work to favor the shorter configuration. Phase plans thus preserve reference-sensitive computation while exposing parallel work, with complete-request costs guiding the choice of execution strategy.

Index Terms—Parallel inference, quantized language models, numerical compatibility, execution planning, speculative decoding.

1 Introduction

Quantized language-model runtimes expose parallel work whenever the next inputs are already known. Prompt processing supplies a sequence of queries; speculative decoding supplies proposed continuations for batched target verification [1], [2]. The execution can use this parallelism without changing the mathematical model, yet produce different floating-point outputs. Batch width can change activation quantization, reduction organization or the physical range consumed by attention. A causal mask fixes which keys are logically visible, but does not fix those numerical operations.

An established deployment needs more than repeatable output: it needs compatibility with the execution already used by regression tests and stateful continuations. That execution includes the prompt producer, incoming KV state and requested output rows, as well as weights and numerical libraries. The optimization problem is to retain their numerical behavior across prefill, continuation, verification and commit while reorganizing known-query work.

There are several ways to control numerical execution. Batch-invariant operators fix reduction choices [3]; arithmetic reconstruction and compiler enforcement can preserve existing library behavior [4]; deterministic engines can relate state invariants to kernel contracts [5]. Fixed-shape schedules and verified speculation provide other system designs [6], [7]. We study compatibility with a quantized deployment fixed before optimization. Parallelism comes from reorganizing independent work around its reference arithmetic.

A *reference-derived phase plan* connects these requirements (Fig. 1). Source analysis and native tests establish an arithmetic recipe; the runtime reads cache occupancy, head and padding to assign each query its physical range. In the illustrated append, the first three queries require extent 768 and the fourth requires 1024. The plan executes these compatible groups while keeping the target and KV resident. Requested vectors remain associated with their logical rows, preserving the frontier needed after a commit or trim.

Two lowerings implement these requirements. Whole-graph lowering places cuts around an entire graph and can realign subsequent groups after a physical boundary. Local-attention lowering applies the cuts within attention, retaining broader projection and feed-forward work. Projection organization is a separate choice. A backend loop reuses a reference-compatible small-column arithmetic path; independent-channel packing executes several such groups together. For the affected CUDA Q8 path, the preserved group has four columns. The mixed Q4_K/Q6_K extension retains scalar-column arithmetic and parallelizes the independent columns. The arithmetic group is type- and backend-specific; the physical-prefix and state interface is shared.

The evaluation covers three model families and both backends. A Linux/Ampere deployment rebuilds the numerical modules and establishes its own reference. Frozen CUDA rules transfer between grouped-query and multi-head attention structures; a typed recipe extension supports a 32B mixed-quantization model. Comparisons stay within each deployment's numerical contract.

On Linux/A5000, compatible speculative applications reduce resident service by 17.21%, 25.54% and 11.52% relative to the fastest measured fixed target-only control for Qwen3-8B, Qwen3-14B and Llama-8B. These are application gains. Balanced factorial experiments isolate packing's 0.80%–1.64% increment and the smaller, geometry-dependent contribution of local cuts. In a 32B target-only deployment, channel grouping saves 5.61% relative to graph-level scalar-column execution.

• *C. Li is with the University of Electronic Science and Technology of China, Chengdu, China. E-mail: chengzhan.li@std.uestc.edu.cn.*

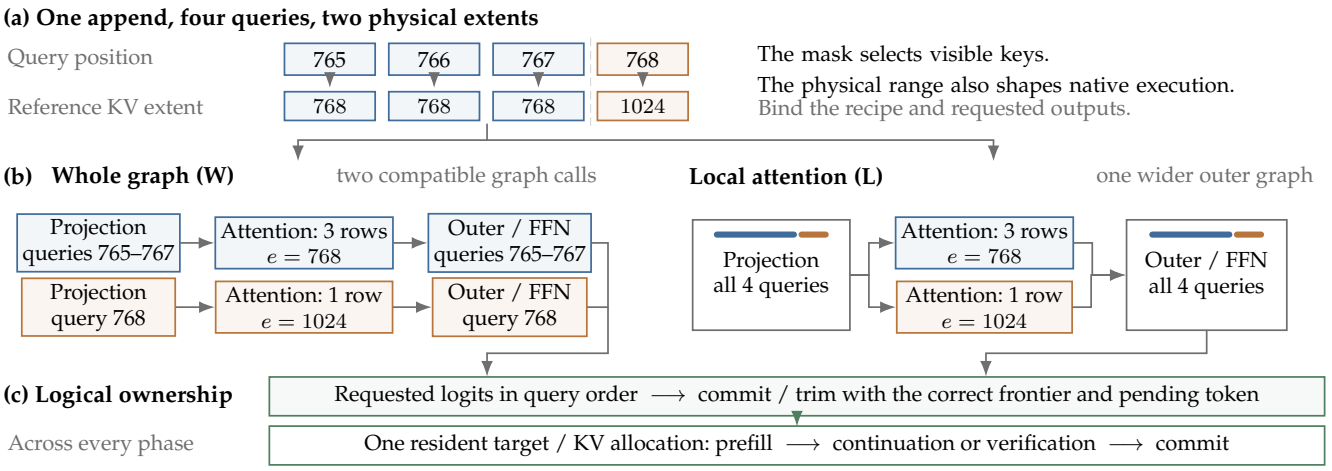


Fig. 1. Reference-derived phase execution for one measured append. (a) Positions 765–768 require extents 768, 768, 768 and 1024. Blue and ochre track the two groups. (b) Whole lowering splits the graph; local lowering confines the split to attention and retains wider outer work. (c) Both preserve logical outputs and the frontier on one resident state. Arrows denote dependencies, not measured durations.

The contributions are a phase-plan interface that composes producer, physical-prefix, arithmetic and state requirements; whole and local implementations with reference-compatible Q8 and mixed-Q4 groups; and an evaluation that separates structural transfer, implementation increments and complete-service value. Ordinary generation and speculative verification use the same interface, with strong simple controls and full request costs determining the useful configuration.

2 Reference Semantics and Physical Execution

2.1 The fixed reference is a deployment execution

Let R_d denote the execution fixed for deployment d before candidate optimization. Its identity includes model and tokenizer bytes, prompt formatting, numerical modules, device placement, KV type and capacity, producer calls, requested outputs, sampling and stopping semantics. A candidate F_d may change how known queries are scheduled while preserving the observations required by R_d.

For a materialized token prefix x, write $\ell^R(x)$ for its full binary32 target vocabulary vector and $K^R(x)$ for its active KV state in canonical logical order. The numerical target at a declared observation boundary is

$$\ell^F(x) \equiv_{\text{bits}} \ell^R(x), \qquad K^F(x) \equiv_{\text{bits}} K^R(x), \tag{1}$$

with matching logical positions, frontier and termination behavior. Equality is checked on complete values, without a tolerance. Active state excludes unused storage, whose bytes need not describe a logical token.

The producer is phase-specific. The CUDA references use a whole-prompt caller with physical microbatch one and last-only prompt output. The Vulkan reference retains its original physical microbatch 128 producer, with the declared outer prompt chunks, then scalar continuation. Replacing that producer with scalar prefill would change the numerical reference. Likewise, a context's output reservation is a capacity declaration, not the number of occupied query rows in a call.

Speculative execution evaluates prefixes that may later be rejected. Each requested target vector is compared with an independent reference forced through the same prefix after the original producer. An autonomous reference, receiving only the original prompt, supplies the emitted-token and stopping oracle. This separates numerical correspondence on a candidate branch from the free-generation endpoint. Draft logits and proposals can change; the target must satisfy the reference contract on the prefixes it actually evaluates.

2.2 The physical range participates in numerical execution

A measured Llama example illustrates why logical equivalence alone is insufficient. Four queries at positions 765–768 required extents $[768, 768, 768, 1024]$ under the single-query reference. An unsplit batch used 1024 for every row. Captured visible Q, K, V, QK and effective probability prefixes were identical, but the AV outputs differed. Cropped-range and padded-range replays reproduced the respective reference and batched results [8]. The earliest saved full-vector difference preceded the first emitted-token difference, at positions 29 and 81 respectively.

The intervention identifies a physical execution requirement. On Vulkan, extent changes continuity and dispatch as well, so the intervention identifies their combined physical effect. A second mechanism concerns activation arithmetic: affected Q8-weight projections use F32 activations on the scalar path, but a wider path can introduce Q8_1 activation quantization. Retaining the F32 path restores the corresponding arithmetic condition. In a Llama control, that repair alone matched emitted tokens while 99 committed full vectors still differed; the physical-prefix repair removed the remaining discrepancies. The mechanisms are separable and have different backend implementations.

The planner derives cuts from live state. At physical head 767, a Qwen witness requires extents $[768, 1024, 1024, 1024]$; the resulting whole-graph $1 + 3$ plan and local-attention counterpart both preserve the independent reference. The same extent rule handles this witness and the $3 + 1$ example in Fig. 1.

TABLE 1
Information bound by a phase plan. Source qualification establishes the arithmetic recipe; live extraction supplies the current geometry and ownership.

Requirement	Effect on execution
Producer and phase	Retain the original incoming state and phase-specific numerical behavior.
Physical prefix	Read occupied slots, head and padding; bind each row's reference extent.
Typed arithmetic	Match activation, dot/reduction, stride, placement and dispatch requirements.
Requested rows	Preserve the original mask and own outputs by logical position.
Commit state	Maintain active KV, trim frontier, pending token and native stopping.
Reuse identity	Include effective geometry and phase policy in graph-reuse checks.

2.3 State and requested outputs connect the phases

The state at a commit contains more than a token list. It includes the materialized prefix, its active KV, the logical frontier vector and any emitted token that has not yet been materialized. A cap or end-of-generation stop can leave the last emitted token pending. Verification can instead append several proposals and then trim rejected state.

Consequently, the last physical output buffer is not always the frontier of the retained logical prefix. The executor must associate requested vectors with their logical rows and preserve the required one before another native call overwrites scratch storage. It must also materialize a pending correction or bonus token exactly when the next phase requires it. These ownership rules make a compatible operator usable in a complete inference transaction.

3 Reference-Derived Phase Plans

3.1 Separate arithmetic qualification from per-call planning

A backend recipe describes the operator conditions that preserve the deployment reference: weight and activation types, reduction organization, supported shapes and strides, placement and output layout. We establish these conditions through source analysis, own-path replay and controlled comparisons with independent reference operands. Descriptor guards select recipes whose numerical correspondence is established separately through native tests.

The planner combines the recipe with sequence ownership, occupied slots, physical head, capacity and padding. In the supported dense, nonwrapping append profile, occupied cells form the logical prefix and the physical head agrees with its end. The extractor checks this correspondence and rejects shifted, wrapped or holey states.

For first physical slot H, query row i, cache capacity N and padding P, this profile gives

$$e_i = \min\left(N, \max\left(P, P\left\lceil \frac{H+i+1}{P} \right\rceil\right)\right). \tag{2}$$

Each row carries its position, e_i, admitted arithmetic class and requested-output flag. The class is normally constant within one attention graph, while the extent changes at a physical boundary. Graph-reuse identity includes the recipe, per-row geometry and phase policy as well as the outer shape.

Algorithm 1 Execute a reference-derived phase

Require: Live target/KV owner S, phase p, known rows Q, original requested-row mask O, admitted recipe a
1: Check phase, capacity, cache ownership and support of a
2: Extract physical-prefix requirements $\{e_i\}$ from S
3: Form compatible runs under (e_i, a)
4: Lower runs to whole-graph calls or local attention views
5: **for** each native call in logical order **do**
6: Execute the selected arithmetic and original output requests
7: Synchronize; store requested vectors by logical row
8: Check successful append and expected physical execution
9: **end for**
10: Commit or trim with the owned logical frontier and pending token
11: On native failure, invalidate the owner for further execution

3.2 Group compatible queries and place the cuts

Let κ_i contain a row's physical extent and recipe class. Whole-graph lowering (W) takes maximal adjacent runs of equal κ_i, subject to the native width bound $W_{\max}$. For constant-key run lengths $r_1, \ldots, r_m$, its minimum number of contiguous calls is

$$G_{\min} = \sum_{j=1}^{m} \lceil r_j / W_{\max} \rceil \,. \tag{3}$$

Every key change forces a cut, and each run needs at least the stated number of bounded calls. Taking the longest allowed prefix of each run attains this grouping bound. Runtime also depends on cut placement and the backend implementation of each group.

Whole lowering executes projection, attention and feed-forward work for each compatible call. After a boundary, it can restart full-width calls rather than preserve the remainder of an arbitrary outer chunk. We retain this rechunking behavior as a strong simple implementation.

Local lowering (L) keeps the outer graph wider and forms compatible groups inside attention. Q, K, V and mask views use the appropriate reference extent. The adapter supplies the strides and, when required, contiguous mask materialization expected by the admitted backend. Attention outputs rejoin in query order, so projection and feed-forward operations outside the cut can use the wider query group. Native attention may further tile a compatible group according to its qualified recipe. Local cuts preserve parallel work around attention; they do not remove dependencies between autoregressively generated tokens.

The grouping pass is linear in the number of query rows. The current dense-cache validator also inspects the occupied prefix. Both extraction and grouping remain in the measured service, together with native dispatch and required synchronization.

3.3 Preserve arithmetic within groups, parallelize between groups

Projection organization is independent of cut scope. Consider $X \in \mathbb{R}^{K\times q}$, containing q query columns, and a shared weight matrix. A recipe specifies an arithmetic group width a whose native computation matches the reference. When $q = ag$, packing exposes the group index as an independent dimension:

$$\widetilde{X}_{k,c,j} = X_{k,aj+c}, \quad \widetilde{X} \in \mathbb{R}^{K\times a\times g}. \tag{4}$$

The backend computes each group with the admitted inner organization; the result is restored to logical column order. Each supported reshape is bound to the tensor layout and backend dispatch validated for that recipe.

3.3.1 CUDA Q8 recipe

For affected Q8_0 projections with F32 input, the implementation preserves the existing small-column MMVQ path with $a = 4$. The backend-loop4 control quantizes the input and invokes the original small-column specialization for successive groups and the supported tail. Packing uses $[K, 4, g]$ views for eligible multi-group work and shares the weight across groups. The four-column reduction within a group is retained. Tails retain the native path validated for their shape.

This distinction matters for verification. A $k = 3$ transaction requests at most four target positions and does not enter the wider packing route. A $k = 7$ transaction can request eight positions and two independent four-column groups. The draft's prefill microbatch, denoted D1 or D4 in the experiments, is a separate parameter.

3.3.2 Mixed Q4_K/Q6_K recipe

The 32B model contains Q4_K, Q6_K and F32 tensors. Its affected mixed-quantized projections retain the original scalar-column path, so $a = 1$. One implementation creates a view, scalar matrix operation and output concatenation for each column. The grouped implementation maps the same columns to $[K, 1, q]$ and executes them as independent channels, then reshapes the output to the caller's layout. The original per-type dot functions, activation quantizer and accumulation organization are preserved. The Q8 four-column guard remains unchanged.

The mixed-Q4 service implementation changes graph lowering and compiled into a new core archive; the recorded CUDA numerical objects remain byte-identical. Qualification uses actual full-model operands and an independent scalar reference, followed by full-model state and output checks. The full-service control is graph-level scalar-column execution using the same per-type arithmetic.

3.3.3 Vulkan mapping

Affected Q8-weight projections retain F32 activation input rather than selecting the wider Q8_1 path. Attention preserves the admitted scalar-equivalent AV reduction and its stride/dispatch requirements, including the p021 mapping used by the resident implementation. The physical-prefix grouping principle is shared with CUDA; its native arithmetic implementation is backend-specific. This supports two-query verification and wider known-query append with the original prompt producer.

3.4 Keep one state owner across phases

The phase interface binds each plan to the existing target context. Prefill, ordinary continuation and verification share one resident target/KV owner; a phase transition changes the execution policy while retaining that allocation.

The caller's requested-row mask is preserved through splitting. After a native call has made the requested vectors available, the executor copies them into logical output slots before subsequent calls can reuse scratch memory. The accepted prefix can then select its frontier even when a later physical row was rejected. A commit or trim updates active state and pending-token bookkeeping together. Reset returns the owner to its declared initial condition; an execution failure prevents further use of a partially updated owner.

Clean execution performs planning, synchronization, output copies and state operations. Independent-reference replay and heavy internal-state capture are confined to qualification runs using the same numerical build and transaction semantics.

3.5 Why the requirements compose

The compositional argument has three obligations. A phase begins from reference-compatible active state and producer history. Each native group uses a recipe that preserves the required outputs and state update for its supported descriptor. Finally, logical output ownership and the transaction rules retain the reference frontier and committed state at the next boundary. Under these obligations, an induction over groups and phase boundaries preserves the contract in Eq. (1).

This argument identifies what must be checked when a recipe or lowering changes. Source analysis, actual-operand comparisons and full-model tests evaluate these obligations in the listed deployments. Descriptor guards enforce the resulting supported domain.

4 Implementation and Evaluation

4.1 Deployments and comparison structure

The implementations extend a pinned llama.cpp/ggml runtime, with commit-bound numerical modules, model bytes, tokenizer and prompt templates [8]. CUDA studies use a Windows RTX 5070 Ti Laptop GPU and a Linux RTX A5000; Vulkan studies use an Intel Arc B570. Models cover Qwen3, Llama and SmolLM2 [9], [10], [11]. Each deployment fixes its own reference before candidate comparisons. The Linux deployment rebuilds the reference and candidate numerical modules from source, including the CUDA, base and CPU backends.

The evaluated state profile is dense single-sequence inference, with F16 KV, greedy output and no FlashAttention. Outer known-query work reaches 32 positions; the later CUDA verification study supports eight occupied query rows. Natural service panels use C1024, and the final Vulkan append interface is also checked at C4096. Table 2 binds the measured profiles, producer semantics and cost windows.

W/L denotes whole/local cuts, and L4/P denotes backend-loop4/packed projections. Q8_0 names weight quantization; q is the number of query rows and k the number of draft proposals. M denotes target-only generation. D4 sets draft prefill microbatch UB4; it does not set k.

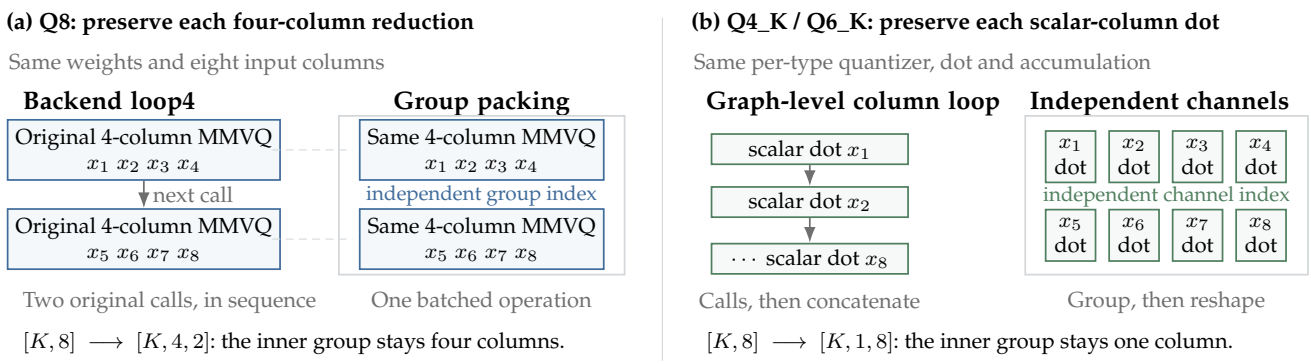


Fig. 2. Arithmetic is preserved within each group. (a) Loop4 invokes four-column groups sequentially; packing exposes them to one batched operation. (b) The mixed-Q4 control uses scalar-column calls and concatenation; grouping exposes the columns as independent channels. Quantizer, stride and dispatch remain bound to the qualified recipe. Layout denotes operation organization, not timing.

Unless stated otherwise, $k = 3$. All applicable controls receive the same legal numerical, residency and graph-reuse improvements. Ordinary generation remains an application alternative and loads no idle draft in the latest Linux panels.

4.2 Independent numerical qualification

Repeated independent reference runs establish the vectors and logical state for the declared inputs before candidate execution. Forced-prefix and autonomous reference runs check candidate branches and free-generation endpoints, respectively. Comparisons cover requested full vocabulary vectors, active KV by layer and position, frontier, output ownership and pending/stop behavior.

Native tests cover producer transitions, inside/crossing prefixes, tails, occupied widths, acceptance followed by rejection, trim, native EOG, cap and reset. Build records bind the complete source dependencies and loaded modules. Heavy observers are checked against the clean executable and remain outside its timed path.

The Qwen-frozen CUDA rules transfer from grouped-query attention with 128-dimensional heads to multi-head attention with 64-dimensional heads. Llama uses its own family reference. The 32B extension checks actual mixed-Q4 operands, 60 geometry cases and all six paths; its reference has reserve 1 and its candidates reserve 32. Supplementary records give the full comparison counts and observation boundaries.

Clean performance runs check reference endpoints, semantic traces, physical work, module identity and the absence of heavy state APIs. Full-vector/state correspondence is established at the declared heavy qualification boundaries; clean timing measures the same numerical implementation with endpoint and trace checks.

4.3 Natural inputs, warmup and service clocks

Natural service inputs come from pinned APPS and MATH snapshots [12], [13]. The Linux Qwen panels reuse the Windows panel's 16 task records, tokenized prompts and cap definitions. Each deployment establishes its own numerical reference and measurements; task reuse does not enlarge the independent sample. Llama uses 12 roots selected after project-exposure exclusion. The Windows $q = 8$ live panel has 16 separate roots, prompt length at most 192 tokens and cap 96. Its equal-work study instead uses one APPS and three GSM8K [14] development origins selected before timing (Supplement S4). The 32B panel uses four exposed mathematics roots with cap 128. The earlier producer study uses GSM8K and MATH-500 (Supplement S5). "New" refers to project exposure, not model-training membership.

In the Linux service panels, each arm/session starts one resident worker, loads its required models and contexts, performs two separate primer requests, and processes the fixed test roots with the declared reset policy. The 32B panel instead uses an identical-root full warmup immediately before each measured request. Both warmup schemes are retained as distinct designs. Each request ends at native EOG or its token cap, preserving its naturally observed output length.

Resident service contains the registered reset, planning, target/draft execution, sampling, required state bookkeeping, synchronization and requested-output materialization. Separate clocks record process startup, model loading, integrity checks, primers, request-burst wall time and wrapper work. These overlapping windows describe distinct lifecycle costs. The first completed response measures full-request latency. Memory is reported from periodic device-global samples.

Vulkan's structural endpoint starts after valid state preparation and includes phase binding, decode, synchronization and output copying. State preparation and import are separately accounted. Equal-work q8 diagnostics likewise retain identical input/state/output contracts and report their operation boundary. Profiling and synchronized host-component diagnostics use separate runs. Primary latencies include every operation within their declared measurement boundary.

4.4 Estimands and preserved dependence

For method A, task root r and registered session s, let t_{rsA} be complete resident-service time. The main application contrast is

$$\rho_{A/B} = \frac{\sum_{r,s} t_{rsA}}{\sum_{r,s} t_{rsB}}. \tag{5}$$

A ratio below 1 favors A, and $1 - \rho$ is the time reduction. We also retain equal-root geometric ratios and per-session summaries, which answer different weighting questions. Tables identify the fastest measured fixed M for descriptive

TABLE 2
Main evidence panels. Counts exclude separate numerical qualification. A repeated timing is not an additional independent task. The 8B and 14B Linux transfer panels share the same sixteen task identities. Source and full control tables are in the supplement.

Panel	Platform / model	Compared paths	Root / repetition unit	Measured + warm	Observation and cost window
Linux service	A5000; Qwen3-8B, 14B	Four target-only M; W-L4-D4, L-P-D4	16 exposed roots; 4 sessions	$384 + 48$ each	Resident service plus actual startup, request burst and worker clocks.
Llama service	A5000; Llama-8B/1B	Same six paths	12 new project roots; 4 sessions	$288 + 48$	Independent family reference; all fixed controls.
Balanced factors	A5000; Qwen3-8B, 14B	W/L $\times$ L4/P; D4 fixed	16 roots; 4 Williams rows	$256 + 32$ each	Cut and projection attribution in the same panel.
Mixed-Q4	A5000; Qwen3-32B	Four M; two D4; REF1 loop/group	4 exposed roots; 4 rounds	$96 + 96$	Resident service; shared graphs-off policy; descriptive.
Eight-query live	Windows RTX; Qwen3-8B	W/L $\times$ L4/P $\times$ k3/k7	16 new roots; 8 Williams rows	$1024 + 128$	Complete service; common candidate output reserve 8.
Equal-work width	Windows RTX; Qwen3-8B	One q8 vs two q4 calls	4 fixed origins; paired cells	$128 + 32$	Identical state, eight tokens and all eight outputs.
Vulkan append	B570; SmolLM2-1.7B	Scalar reference, W, L	12 structural cells; 3 repeats	$108 + 108$	C1024/C4096; append after valid state preparation.

comparison; the registered inference compares the candidate with every fixed M, rather than testing an outcome-selected minimum.

The Linux application panels use 10,000 paired-root bootstrap draws, retaining every arm and session of a sampled root. Five fixed comparisons use 99% percentile intervals with Bonferroni adjustment. The two balanced factors panels similarly use five estimands: four direct contrasts and an equal-root multiplicative interaction. The four Williams rows balance four-arm positions and within-session directed neighbors. By contrast, the six-arm service designs have partial order balance; the first Linux 8B D4 comparison retains a fixed relative order. We therefore use the balanced panels for cut/projection attribution.

The Windows q8 panel uses eight Williams order blocks. Its four primary width comparisons use 98.75% intervals; the four secondary fixed-k7 comparisons form a separate labeled family. All intervals condition on the finite roots, implementation and observed sessions. Repetitions do not increase the independent root count. The four-root 32B panel and deliberately chosen structural diagnostics remain descriptive, with complete observations rather than population confidence claims.

5 Results

5.1 Reference-compatible execution provides complete-service value

Figure 3 shows every fixed target-only comparison for the three Linux deployments. LOCAL-packed D4 reduces resident service relative to all four M implementations. Against the fastest measured fixed M, the reductions are 17.21% for Qwen3-8B, 25.54% for Qwen3-14B and 11.52% for Llama-8B. The corresponding average request times change from 2.6898 to 2.2269 s, 4.6022 to 3.4268 s and 2.8515 to 2.5230 s. The adjusted intervals support the direction of each registered M comparison.

The 14B deployment also retains the benefit outside the internal service timer. The actual measured-request burst totals 294.632 s for its strongest fixed M and 219.405 s for LOCAL-packed D4, a 25.53% reduction. Complete native-worker totals, including integrity checks, loading, primers and the request strings, change from 662.546 to 594.559 s, a 10.26% reduction. For Llama, resident requests benefit, while the recorded process-level total favors M for this short worker workload. Table S2 reports resident and full-worker costs separately.

The Windows practical panel originally selected 16 task roots and retained all four target-only controls. LOCAL-packed D4 reduces resident service by 29.11%, actual request-burst time by 29.09% and worker-wall time by 26.00%. Its complete phase account locates the savings primarily in continuation/verification. Supplement S2 gives the absolute times and disjoint cost balance. The Linux Qwen panels reuse the Windows task records, while numerical references, measurements and deployment lifecycles are established separately.

Compatible target verification saves enough target work to cover draft work in these deployments. This gain belongs to the reference-compatible application, which whole-graph execution also supports. The balanced studies resolve the incremental effects of the two implementation choices.

5.2 Balanced studies separate the sources of improvement

The four-D4 factorial experiments hold the model, draft policy and transaction width fixed, then cross W/L cuts with loop4/packing. Every arm is remeasured in the same panel. Figure 4 includes the root-level ratios and all four direct effects for both Qwen scales; the fifth member of the

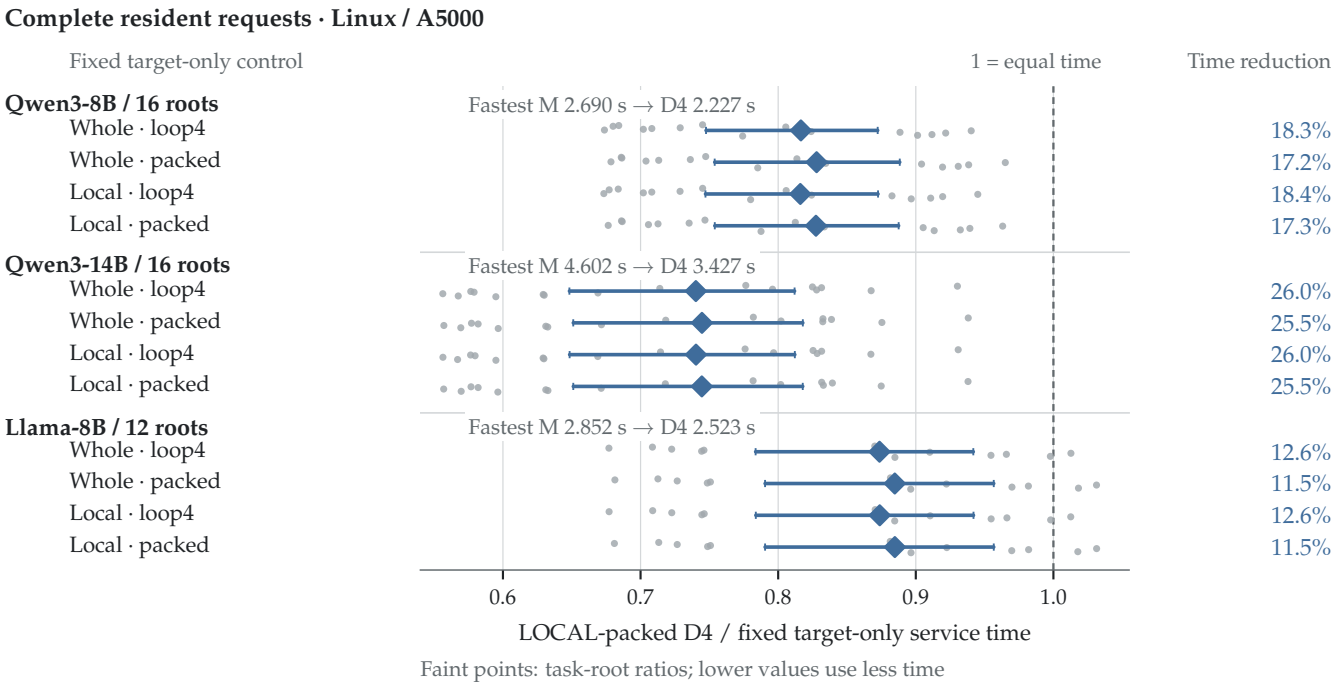


Fig. 3. Linux/A5000 resident service. Faint points show all task-root ratios. Each diamond compares LOCAL-packed D4 (draft prefill UB4, $k = 3$) with a fixed target-only M; segments are 99% paired-root intervals from the five-contrast family (the four M comparisons shown here and the W-L4-D4 comparison). Headers give fastest fixed M and D4 mean times. Qwen panels share 16 tasks; Llama has 12. Deployments are analyzed separately. Table S1 reports all six arm means.

simultaneous-inference family is the root-level interaction reported in Supplement S3.

At 8B, packing reduces service by 1.64% with W and 1.56% with L; at 14B, it saves 0.99% and 0.80%. Backend-loop4 already moves the simple loop out of the graph, so these comparisons measure the remaining packing increment.

At fixed loop4, local cuts save 0.21% and 0.30% for 8B and 14B. At fixed packing, the point reductions are 0.13% and 0.11%, with intervals containing 1. The interactions, 1.000696 and 1.001703, indicate mild attenuation when the two optimizations are combined.

The physical execution explains the scale of the cut effect. In the 8B factorial, 20 of 2,224 logical verification cycles require an extra whole-graph boundary split, approximately 0.899%. LOCAL executes 2,224 four-row graphs and W executes 2,244 verification graphs, with the same number of target rows and the same prefill-width distribution. Local grouping avoids repeated graph work at those boundaries; most calls already have compatible geometry. The native observer places wide packing in prefill, not in four-row verification. This phase distinction explains where packing can contribute at $k = 3$.

Packing organizes independent arithmetic groups, while local cuts preserve outer parallelism when a physical boundary divides a call. A deployment selects the implementations; per-call planning supplies their compatible groups.

5.3 Typed grouping extends the method to 32B mixed quantization

The 32B deployment contains 385 Q4_K, 65 Q6_K and 257 F32 tensors. Its REF1 recipe is a typed extension: it retains scalar-column arithmetic for the mixed quantized projections and exposes independent columns as channels. All six paths pass finite vector/state checks before the four-root descriptive timing panel. The candidates share C1024, F16 KV, reserve 32 and a CUDA-Graphs-disabled policy. Transformer and output layers are GPU offloaded, with the standard CPU-mapped input embedding.

Grouping reduces ordinary service from 124.183 to 117.208 s with W fixed (5.62%), and from 124.100 to 117.144 s with L fixed (5.61%), over 16 requests per arm. Both comparisons improve in all 16 matched root/round pairs. Worker totals fall by 3.98% and 4.56%, and caller totals by 3.85% and 3.81%, respectively. The grouped implementation retains scalar-column arithmetic while replacing separate column graphs and concatenation with one operation over independent channels.

Figure 5 identifies target-only grouped execution as the appropriate measured application choice. The grouped D4 path improves on the graph-loop D4 path, but takes 14.00% more resident time than the fastest M. The ordinary-generation comparison fixes cut scope and excludes draft work, isolating the measured grouping package. A subsequent operand-level comparison with a backend-internal loop is reported in Supplement S3.

5.4 Physical geometry determines the value of local cuts

The final Vulkan interface supports actual C1024 and C4096 allocations, the original prompt producer and known-query appends of 2, 4 and 32 rows. Figure 6 retains both within-bucket and crossing cases. In the six crossing coordinates, LOCAL reduces append time by 20.34%–44.68% relative

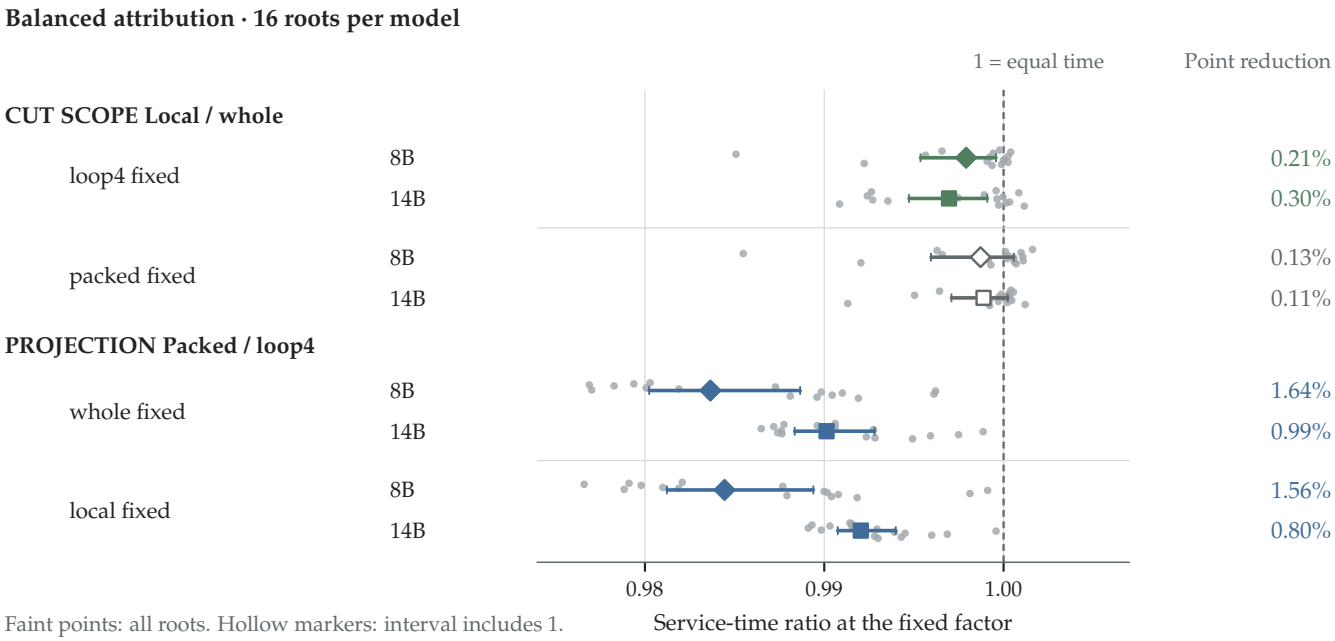


Fig. 4. Balanced D4 attribution. Faint points show all 16 root ratios. Diamonds (8B) and squares (14B) mark complete-service total ratios; segments are 99% adjusted intervals. Hollow, neutral markers identify intervals that include 1. The two sections isolate cut scope and projection. Table S3 lists the fixed effects; Supplement S3 defines the equal-root interaction.

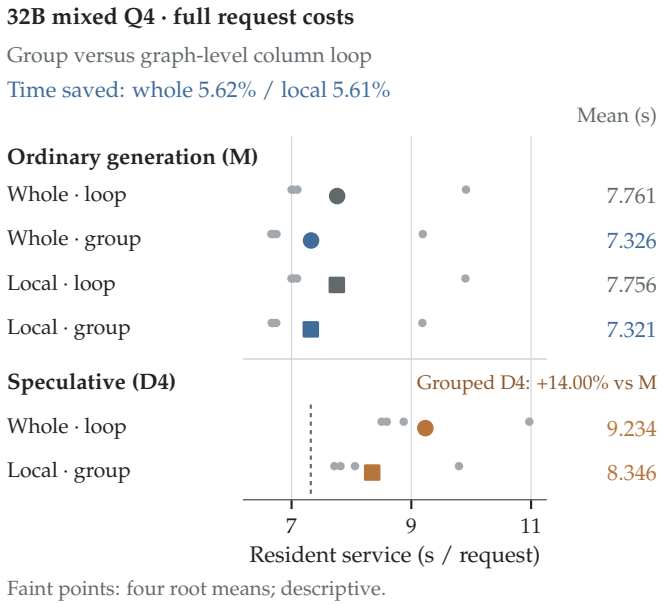


Fig. 5. All six 32B mixed-Q4 paths. Markers and numbers give mean service; faint points are four exposed-root means over four rounds. Circles/squares distinguish whole/local scope. The loop control is graph-level scalar-column execution. Grouped D4 uses 14.00% more time than the fastest M (dotted reference). Table S4 reports complete costs.

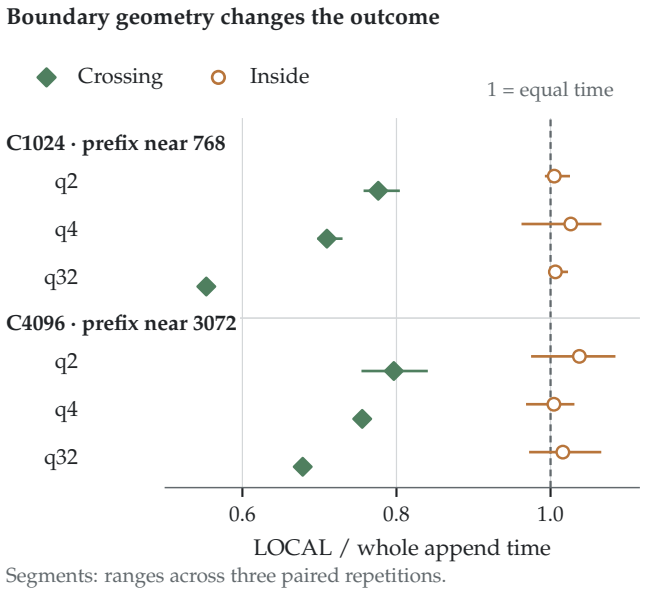


Fig. 6. Vulkan append geometry. Filled diamonds denote crossings, hollow circles the inside controls, and the dashed line equal time. Segments show the minimum and maximum of the three observed paired ratios, not a population interval. Crossings split one row from the remainder. C1024 and C4096 jointly change capacity and active-prefix region.

to W. The paired inside controls are within approximately 0.43%–3.74% in the other direction.

Split geometry determines the work preserved by a local cut. Earlier q32 experiments used $16 + 16$ whole-graph splits and included cases favoring W; the final interface probes a $1 + 31$ split, one row before a physical boundary. The earlier full CUDA width/geometry matrix is reproduced in Supplement S5. Its twelve fixed coordinates include cases where packing regresses, including Qwen crossing widths.

The historical Vulkan cells are retained in Supplementary Fig. S2. Natural alignment and whole-graph rechunking can make the schedules nearly coincide. Local grouping provides its benefit where it avoids fragmentation of the surrounding graph.

Vulkan executes two-query verification within the reference contract. In a separate resident mechanism window, LOCAL executes 578 occupied two-row target graphs for 1,156 rows, versus 1,156 scalar graphs in the earlier resident speculative path. Ordinary M executes 992 graphs for 992 rows. Batching reduces graph invocations while speculation changes the amount of target work. The complete pilot cost, including draft work, favors M in that deployment. The earlier natural-prefix AUTO panels on both RTX and B570 did not establish a service benefit over fixed policies; the strong whole-graph rechunk control can absorb much of the append-only gain. Supplement S5 retains those separate negative controls and their lifecycles.

5.5 Eight-row execution improves equal work; live width changes work

The $q = 8$ implementation retains Q8_0 weights and the arithmetic reference, extending the transaction to $k = 7$ with candidate output reserve 8 for both widths. The width is physical: LOCAL can execute eight verification positions, while W splits according to actual extents. Original requested rows, pending/correction/bonus state, trim frontier and terminal handling are qualified against the independent reference.

Across the four development origins, one $q = 8$ operation uses 3.94%–11.91% less aggregate time than two $q = 4$ operations in the four RTX cells (Fig. 7a). Individual origin ratios remain visible, including the loop4-crossing origin above 1. The separate A5000 live pilot is reported in Supplement S4.

In live generation, k7 reduces cycles from 3,504 to 2,296 per arm but increases verification rows from 14,016 to 18,368 for the same 11,472 emitted tokens. Acceptance among inspected proposals is 87.47% and 87.93%; acceptance among all proposals is 75.42% and 56.94%. The separate proposal accounts in the supplement distinguish actual mismatches from unconsumed suffixes.

All four complete-service width comparisons favor k3 by the ratios shown in Fig. 7b. For LOCAL-packed, $k = 7$ adds 24.897 s over 128 requests: verification adds 13.567 s, draft proposal adds 13.224 s, and draft synchronization saves 2.792 s. Output copying contributes 0.252 s; the remaining work closes the balance. Supplementary Fig. S1 and Table S6 preserve the decomposition and all eight arms.

The complete cost can be written as

$$T_{\text{spec}} = T_{\text{target}} + T_{\text{draft}} + T_{\text{state/output/host}}, \tag{6}$$

with disjoint intervals and a reconciled residual. Wider execution reduces some per-row cost, while longer proposals alter the total rows and consumed fraction. The evaluated live workload uses k3 as its effective fixed configuration; q8 remains a supported parallel execution capability with its own equal-work evidence.

6 Related Work

6.1 Preserving arithmetic across parallel execution

Thinking Machines studies batch-invariant reduction choices for normalization, matrix multiplication and attention [3]. Taming reconstructs the bitwise arithmetic of existing cuBLAS kernels, enforces reduction descriptions during lowering and verifies compiled arithmetic properties [4]. These works establish that existing numerical behavior can be an optimization target. Our contribution connects reference-compatible arithmetic to dense-cache prefixes, phase transitions and requested-output ownership in a quantized inference runtime. The native grouping recipes expose independent work while retaining the reference-sensitive organization within each group.

Public `llama_reproducible` code includes a small-column quantized execution path [15]. Backend-loop4 is therefore a direct performance control. The balanced experiments separate its remaining projection increment from cut scope. The mixed-Q4 study uses a scalar-column recipe and a different graph-level baseline, as summarized in Fig. 5.

6.2 System determinism and speculative execution

Vosti studies engine-level determinism through formal relationships between engine state, cache behavior and kernel properties [5]. We study compatibility with an existing quantized deployment: phase-specific producers, physical cache geometry and qualified arithmetic determine the whole/local execution plans. Native numerical correspondence and complete-service measurements evaluate these plans.

LLM-42 uses a consistent verifier execution with verification and state repair [7]. Our supported paths organize compatible work before execution by binding offline-qualified arithmetic and cache requirements at each call. CoRun isolates prefill and uses fixed-shape batched decode for deterministic service [6]. Phase plans instead expose row-dependent physical-prefix and output-state requirements, allowing attention-local cuts within a wider graph.

Speculative decoding and sampling reduce target invocations by evaluating proposals together [1], [2]. Distribution preservation does not establish byte-identical finite-precision vectors and KV; the target-compatibility contract specifies those additional obligations. Draft width remains an application control. HeteroSpec allocates verification effort using contextual heterogeneity [16], a different objective from compatibility with a fixed deployment.

7 Conclusion

Reference-derived phase plans connect a deployment's numerical behavior to its parallel execution. They combine the producer history, physical KV range, type-specific arithmetic and ownership of logical outputs and state. Whole-graph and local-attention implementations place the cuts differently while preserving the reference-sensitive computation within each group.

CUDA and Vulkan studies support this approach across model structures, operating environments and quantized paths, including mixed-Q4 at 32B. Controlled comparisons separate packing's contribution from geometry-dependent

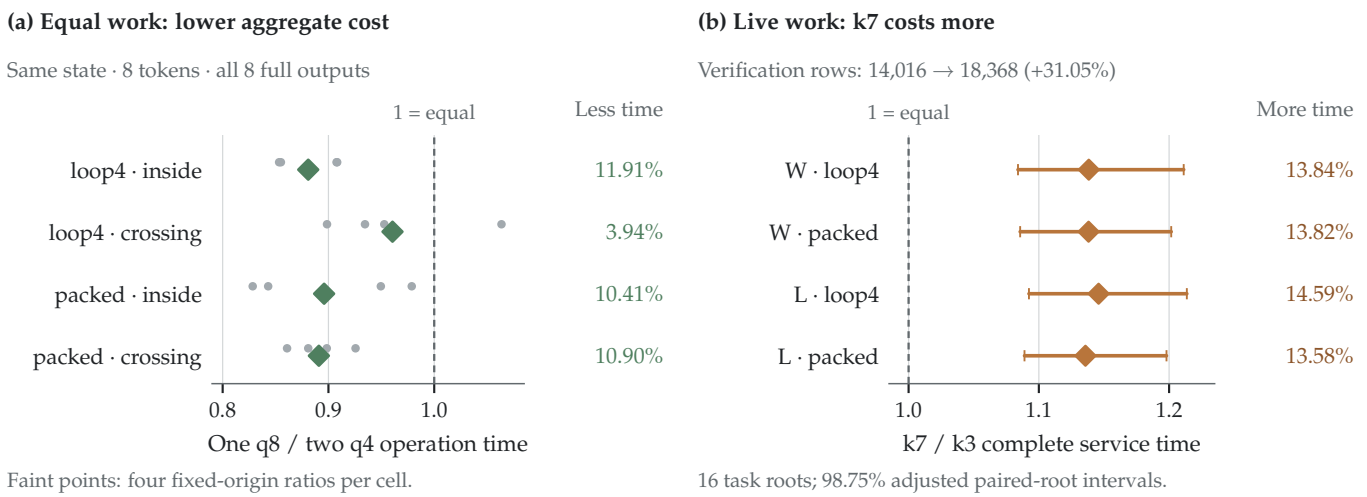


Fig. 7. Equal-work efficiency and live workload. (a) One $q = 8$ call versus two $q = 4$ calls from the same state, using eight known tokens and all eight full outputs. Diamonds and percentages summarize total operation time; faint points retain all four origin ratios (one APPS, three GSM8K). (b) Four fixed-method $k = 7/k = 3$ service ratios from the separate 16-root live panel, with 98.75% adjusted intervals; percentages denote increases. D4 is fixed; no contemporaneous M is timed.

local cuts. Complete requests then determine the useful deployment: speculative execution improves the three Linux service panels, grouped ordinary generation is preferable in the tested 32B configuration, and shorter proposals remain preferable in the live width study despite the aggregate equal-work advantage of eight-query execution. Phase plans make these choices available within an established numerical reference.

Acknowledgments

OpenAI ChatGPT and Codex assisted with implementation, experiment and analysis tools, manuscript editing and figure code. The author is responsible for scientific interpretation and final approval.

References

[1] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 19 274–19 286. [Online]. Available: https://proceedings.mlr.press/v202/leviathan23a.html

[2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," arXiv:2302.01318, 2023.

[3] H. He and Thinking Machines Lab, "Defeating nondeterminism in LLM inference," Sep. 2025. [Online]. Available: https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/

[4] Z. Yang, N. J. Riasanovsky, W. Deng, and V. Sarkar, "Taming bitwise behavior in GPU kernels with tensor core: Black-box reconstruction, compiler enforcement, and static verification," arXiv:2609.11356, 2026.

[5] J. Qin, A. Du, D. Zhang, M. Lentz, and D. Zhuo, "Vosti: Specifying, implementing, and verifying deterministic LLM inference," arXiv:2609.38981, 2026.

[6] S. Zhao, J. Yang, Q. Chen, J. Hu, J. Zheng, G. Chen, and X. Chen, "CoRun: Padding is simple and efficient for deterministic LLM inference," arXiv:2608.14376, 2026.

[7] R. Gond, A. K. Kamath, R. Ramjee, and A. Panwar, "LLM-42: Enabling determinism in LLM inference with verified speculation," arXiv:2601.17768, version 2, 2026.

[8] C. Li, "Reference-compatible parallel execution: implementation and commit-pinned companion evidence," 2026, evidence locations and access scope are listed in the supplementary material. [Online]. Available: https://github.com/leecdiang/HeteroSpec

[9] A. Yang *et al.*, "Qwen3 technical report," arXiv:2505.09388, 2025.

[10] A. Grattafiori *et al.*, "The Llama 3 herd of models," arXiv:2407.21783, 2024.

[11] L. Ben Allal *et al.*, "SmolLM2: When smol goes big—data-centric training of a small language model," arXiv:2502.02737, 2025.

[12] D. Hendrycks, S. Basart, S. Kadavath *et al.*, "Measuring coding challenge competence with APPS," arXiv:2105.09938; NeurIPS 2021, 2021.

[13] D. Hendrycks, C. Burns, S. Kadavath *et al.*, "Measuring mathematical problem solving with the MATH dataset," arXiv:2103.03874; NeurIPS 2021, 2021.

[14] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," arXiv:2110.14168, 2021.

[15] amiller and contributors, "llama_reproducible: deterministic quantized inference implementation," 2026, source snapshot f0dafdbd9dba40bd0e596f74f3da5f592900fca4, accessed Nov. 6, 2026. [Online]. Available: https://github.com/amiller/llama_reproducible/tree/f0dafdbd9dba40bd0e596f74f3da5f592900fca4

[16] S. Liu, Y. Ye, Q. Zhu, Z. Cao, and Y. He, "HeteroSpec: Leveraging contextual heterogeneity for efficient speculative decoding," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Jul. 2026, pp. 12 930–12 947. [Online]. Available: https://aclanthology.org/2026.acl-long.589/